Operating System

Index

(Power-on-self-test)

* POST : Post is a process that runs immediately after the Computer is powered on to check whether hardware Components are working properly.

⇒ Why it's needed :
- To make sure hardware is ok before loading the os.
- To detects errors early (RAM, keyboard, cpu, disk).
- Without Post, the os might load on faulty hardware, causing crashes.

⇒ Behind the Scenes : (how it actually works) :
- you press the Power button.
- CPU starts executing code from BIOS (stored in Rom) "read-only memory"
- BIOS runs POST
- Post checks :
  - RAM
  - CPU
  - Keyboard
  - Basic motherboard components.
- if everything is fine → move to booting
- if error : system beeps or shows error.

* BIOS (Basic input/output System) : BIOS is a firmware stored in Rom (read-only Memory) that starts the computer, runs POST, and helps load the os. and contains all factory-default settings

⇒ why it's needed : • cpu needs instruction immediately after power on.
- BIOS : initializes hardware, Runs Post, finds the boot device

2

⇒ Behind the scenes (how it actually works):

- POWER ON → CPU starts
- CPU goes to a fixed memory
- That address points to BIOS in ROM
- BIOS starts running
- BIOS runs POST
- BIOS reads CMOS settings (boot order, time, hardware info)
- BIOS initializes devices (keyboard, screen, disk)
- BIOS searches for a bootable device
- BIOS hands control to the bootloader.

→ Bootable device : A device which has bootloader + OS files. BIOS cannot load the OS directly it needs bootable device.

The bootloader loads the OS into ram and load them to run. So the bootloader acts as bridge b/w OS and BIOS.

* CMOS : CMOS is a small memory chip that stores system settings used by the BIOS.
Customized

⇒ Why it's needed :

- BIOS needs saved information everytime the computer starts
- Example :
  - Date and time
  - Boot order (which device will boot first)
  - Hardware Configuration
- Without CMOS, BIOS would forget settings after power off.

Stored data of BIOS and CMOS

BIOS Stores	CMOS Stores
• A program (firmware) • instructions to : • Start the computer • Run POST • Read CMOS • initialize hardware • Find bootable device • Load bootloader	• Date and time • Boot order (USB, SSD, HDD, CD). • Hardware configuration • Enabled / disabled devices • BIOS setup changes made by user.

⇒ Behind the Scene (How CMOS actually works) :

- CMOS is not a program.
- It only stores data.
- It uses a small battery (CMOS battery)
- when power is OFF :
  - Battery keeps CMOS data safe
- when power is on :
  - BIOS reads settings from CMOS
  - uses them to start the system correctly.

* Booting process : The booting process is the sequence of steps by by which a computer starts and loads the operating system after power on.

Booting makes the system ready for user use.

4

⇒ Behind the Scene (very clear flow):

Power on

↓

BIOS starts (from ROM)

↓

POST (hardware check)

↓

BIOS reads CMOS settings

↓

BIOS initializes devices

↓

BIOS finds bootable device

↓

Bootloader loads OS kernel

↓

Operating system starts.

* Operating System :- An operating system is a system software that controls computer hardware and provides services to user programs.

in simple words:

OS acts as an interface b/w the user and the computer hardware.

⇒ Why it's needed :

- User cannot use hardware directly.
- Programs cannot run properly.
- No file handling, memory use, or device contrs.

⇒ OS is needed to :

- Run programs
- Manage memory
- Control CPU
- Handles files and devices.

⇒ Behind the scenes (how OS actually works):

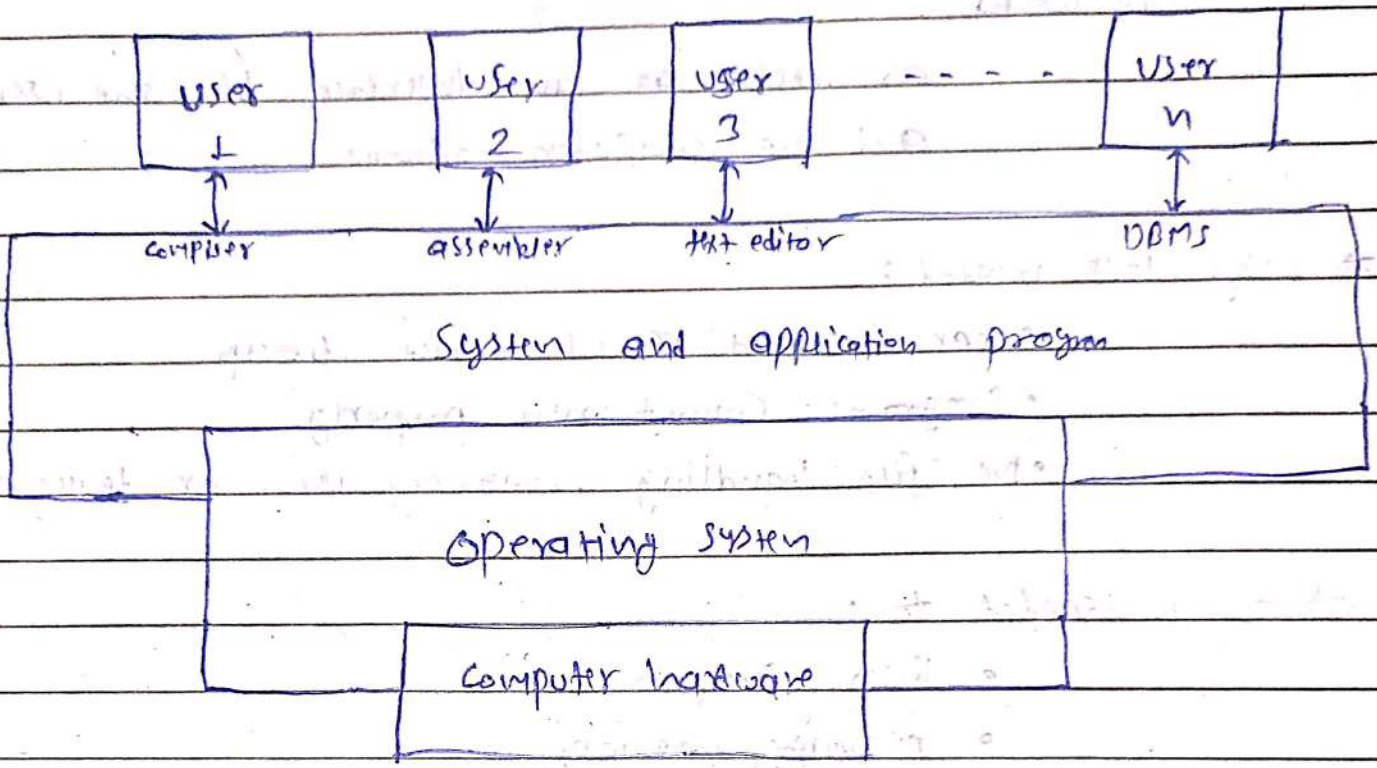
when OS starts :

- It loads into main memory (RAM)
- takes control of :
  - CPU (decides which program runs).
  - memory (who gets how much RAM)
  - Devices (keyboard, disk printer).
- It works silently in the background while you use apps.

→ you never sees it but everything depends on it.

* Computer system ~~Art~~ Structure :

⇒ Components :



- Hardware : provides basic computing resources
  → CPU, memory, I/o devices

- operating System : Controlls and coordinates use of hardware among various application and users

- Application programs : The software which are used by user to solve problems using System resources.
  → word processor, browser, compiler, database, games.

- Users : users are entities the use the computer system, such as people, machines, or other computers.

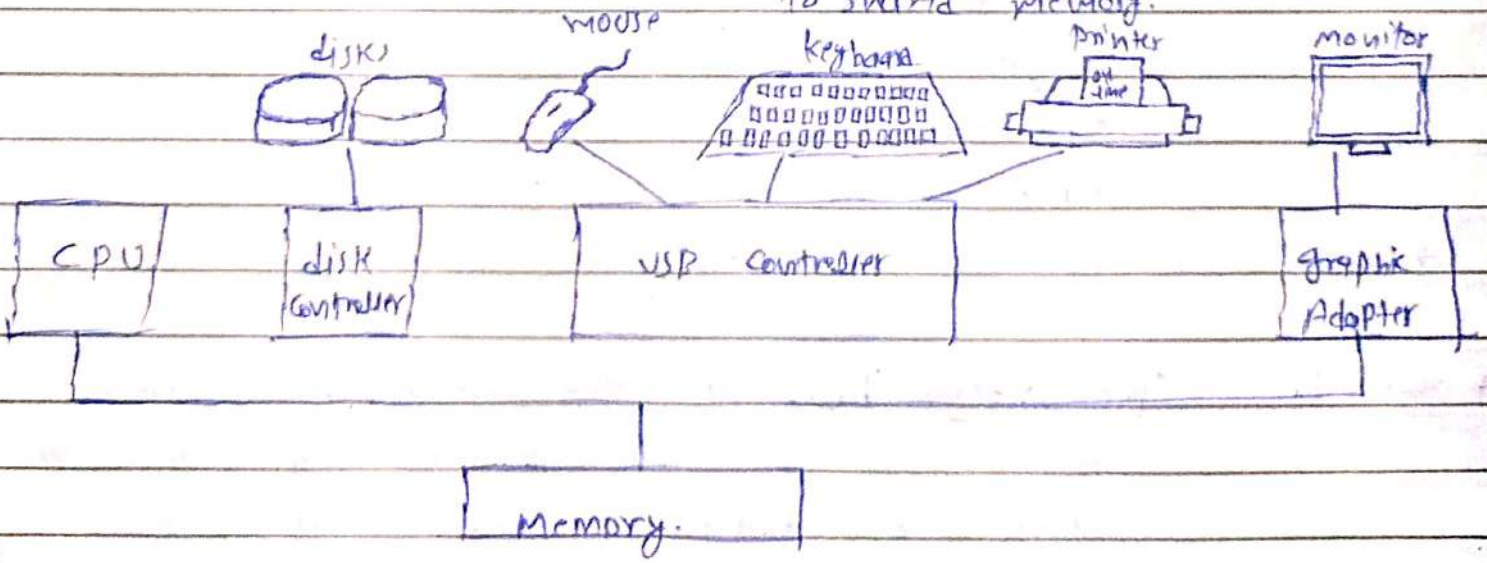
* **Kernel:** The kernel the the core (heart) of the operating system that direcly interacts with hardware and manages system resources.

⇒ Behind the scenes: (how it actually works):

- Kernel runs in kernel mode (full hardware acess).
- It handles:
  - process scheduling (who get the cpu).
  - Memory management (RAM allocation).
  - Device control (keyboard, Disk, printer).
  - System calls from the applications.

- When an app needs hardware:
  - App → System call → kernel → hardware.

* **Computer system organization:** one or mor cpu's, devices Controller Connect Through Common bus providing acces to Shared memory.



8.

* **Device Drivers:** A Device Driver is a Software Component that allows the operating System to Communicate with hardware devices

⇒ Why it's needed:

- Hardware devices are all different.
- OS cannot know how to control every device directly.
- Device drivers:
  - translates OS commands into device specific actions.

⇒ Behind the scene (How it works):

- Application request I/O through OS
- OS passes request to device driver
- Device Driver:
  - Sends proper instructions to hardware
  - Receives responses from device.
- Kernel uses drivers to control:
  - printer
  - Disk
  - Keyboard
  - Network Card

→ Driver works as a translator.

* **System bus:** The System bus is the communication medium through which the CPU, memory and I/O devices exchanges data, addresses, and control signals.

* Device Controller : A Device Controller is a hardware component that control a specific I/O device and manages communication b/w the CPU/OS and that device.

in simple words!

"It takes commands from Driver and then generates interrupt."

⇒ Behind the scenes!

- Device Controller is attached to a device (disk, keyboard, printer).
- It contains :
  - Registers (data, status, control)
  - Local buffer (temporary storage).
- Flow :

Application (Sends command)

↓

Operating System (Kernel)

↓

Device Driver (Software)

↓

Device controller (hardware)

↓

operates the I/O device

↓

I/O device generates interrupt

↓

Sends to Kernel back.

* Interrupt : An interrupt is a signal sent to the CPU to stop its current work and handle an important event.

⇒ Behind the Scene :-

1. CPU is executing a program
2. A event occurs (I/O completed, key presed)
3. Device controller sends an interupt
4. CPU :
   - Saves current state
   - Stops current execution
5. CPU jumps to "interupt Service Routine (ISR)"
6. ISR handles the event
7. CPU Restore the State
8. CPU resumes previous work

⇒ "Every action/event happens on ~~system~~ even in hardware or software generates an interupt."

* Interrupt Service Routine (ISR) : An interrupt Service Routine is a special function that executes when an interrupt occurs to handle that interupt.

⇒ why it's needed :

- interupt tells CPU : "Something happend"
- ISR tell CPU : "Here's what to do about it."
- without ISR, CPU wouldn't Know how to respond

⇒ Behind the Scene :

- When an interrupt occurs :
  - CPU Jumps to ISR address
  - ISR :
    - identify interrupt source
    - Performs required action
  - ISR finishes
  - CPU work on previous task.

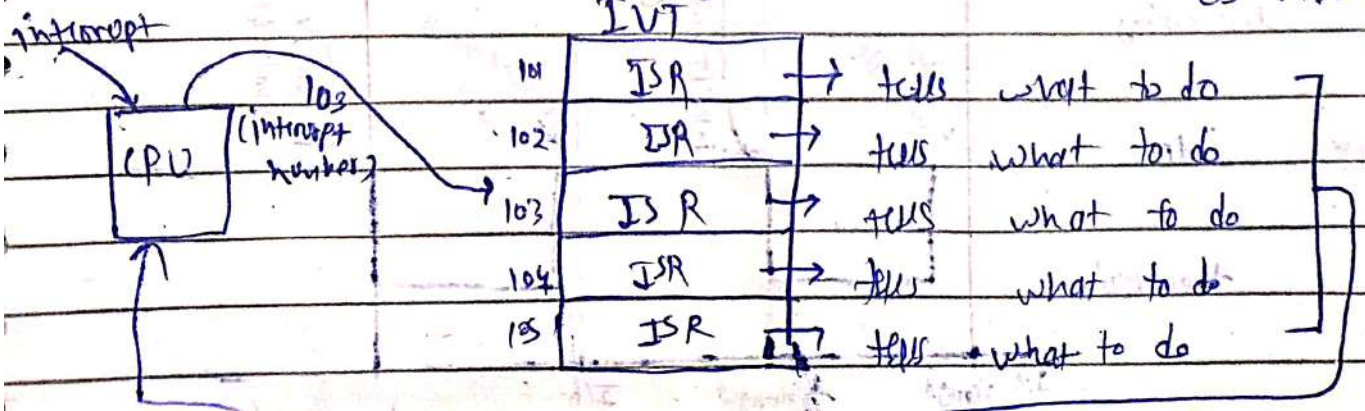
→ "The ISR gives roadmap / process that what to do of interupt ".

* Interrupt Vector : An interrupt vector is an address (pointer) that tells the CPU where the ISR (interrupt Service Routine) is located.

⇒ Behind the scene :

- Each interrupt has a unique number (interrupt number).
- CPU uses this number to index in "interrupt vector Table"
- IVT stores addresses of ISRs
- CPU :
  - Gets interrupt number
  - Looks up ISR address in interrupt vector table (IVT)
  - Jumps to that address.
  - Executes ISR.

(Stored in RAM and regenerated when OS reboots.)



12

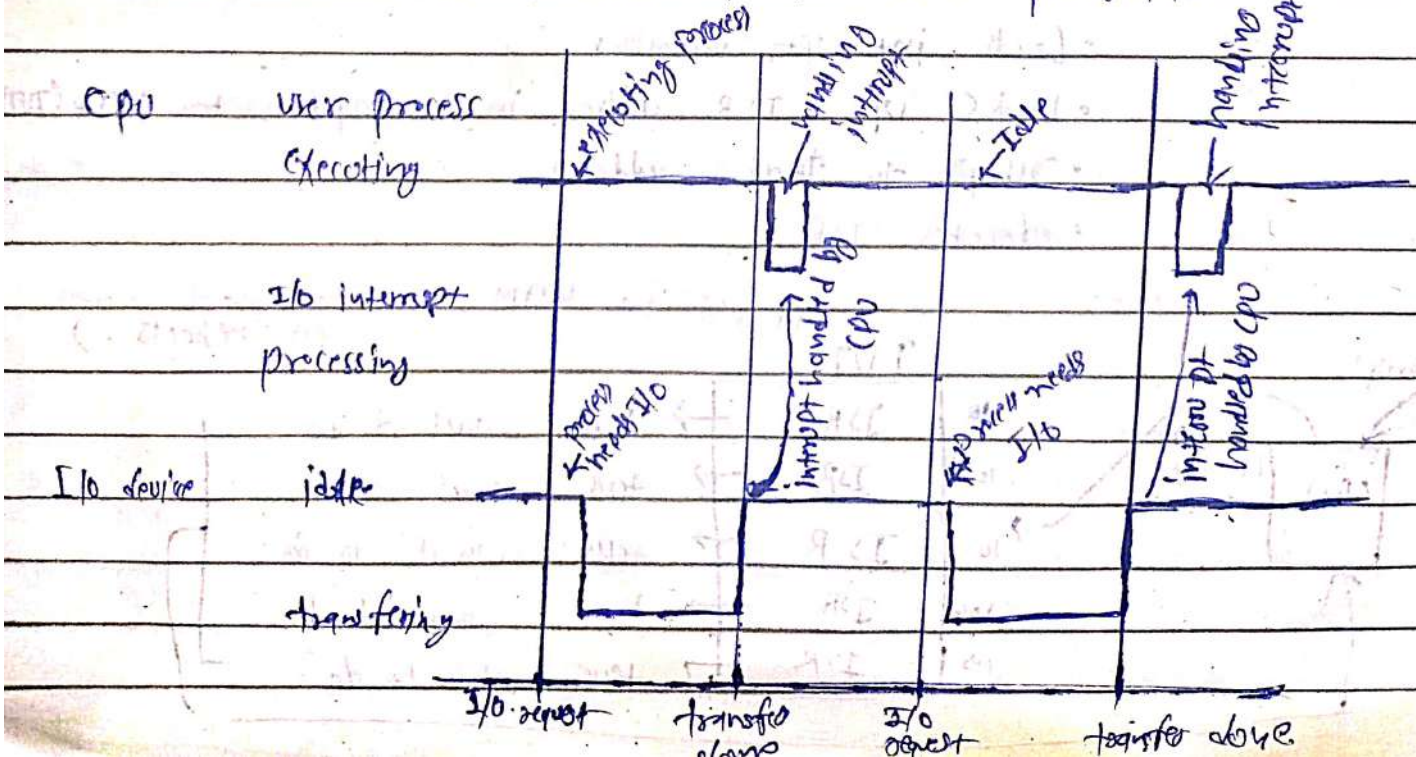
* Trap : A trap is a software generated interrupt that occurs intentionally when an program requests a service from the operating systems or when an error happens.

⇒ Why it's needed!

- Software cannot access hardware directly
- They need a safe way to request OS service.
- Trap Allows:
  - System calls
  - Error handling (divide by zero / invalid memory access).

⇒ Behind the scene!

1. use program executes a special instruction
2. This instruction cause a trap
3. CPU switches from user mode → kernel mode
4. Control jumps to trap handler (ISR)
5. OS performs the requested service
6. Control returns to the user program.



* Polling : polling is a method where the CPU repeatedly checks each device to see whether it needs attention.

⇒ Why it's needed:
- To Know which device wants service.
- Used in simple Systems
- No interrupt support required.

⇒ "polling is mostly used for the devices which doesn't support interrupts like old devices or hardware".

⇒ Behind the Scenes (How it works):
1. CPU finishes its current task
2. CPU checks device 1 : "Do you need service"
3. CPU Checks device 2
4. CPU Checks device 3
5. If a device says yes → CPU service it
6. if no device needs service → CPU keeps checking again.

↳ CPU is actively involved all the time.

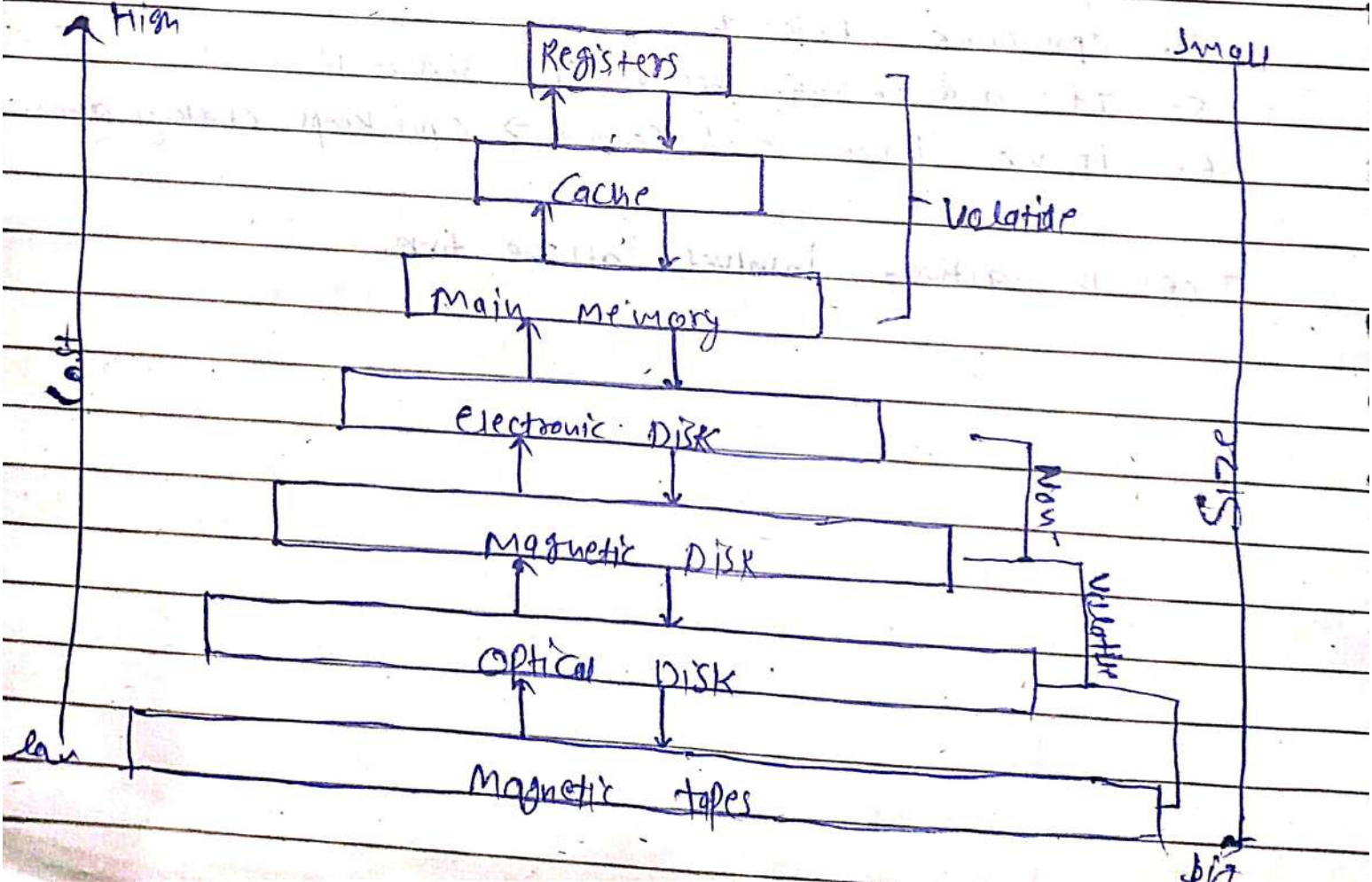
* Storage structure : Storage structure describes how data is stored and organized in different types of memory and storage devices in a computer system.

⇒ Why It's needed :

- Different data needs different speed and size.
- Fast storage is small and costly.
- Large storage is slow and cheap.
- OS must decide :
  - what to keep close to CPU
  - what to keep far away.

⇒ Storage structure is the ~~hierarchy~~ hierarchical organization of memory and storage devices based on :

- Speed
- Size
- Cost



* Volatile Memory : Volatile memory is a type of memory that loses its data when power is turned off.

* Non-volatile memory : Non-volatile memory is a type of memory that retains its data when power is turned off.

(1) Register : a volatile storage that is very small, very fast memory located inside the CPU.

→ We use it to store data and instructions that the CPU is using right now, so execution is extremely fast.

(2) Cache : Cache is a volatile memory which is small very fast memory placed b/w RAM and CPU.

→ We use it to store frequently used data and instructions, so the CPU doesn't have to wait for slower RAM, making program faster.

(3) Main memory (RAM) : A volatile primary working memory where the currently running programs and data are stored.

→ We use it so the CPU can quickly access programs and data while they are executing.

(4) Electronic Disk : It is a non-volatile Storage device stores data using electronic circuits (flash memory) instead of moving parts.

→ Chips, cards.

(5) Magnetic Disk : A non-volatile storage device that stores data on a ~~rotating~~ rotating disk coated with magnetic material.

→ used to store large ~~data~~ amount of data parmanentey. at a low cost (example: OS, files, videos).

(6) optical Disk : A non-volatile storage device that stores data using laser technology on a flat, circular disk.

→ used to store and distribute data like software, movies, music and backups.

→ examples: • CD • DVD • Blue-Ray.

(7) Magnetic tapes: A non volatile storage device that stores data on a long magnetic ribbon (reel).

→ used to store very large data backups at low cost for a long term storage (archieves).

* **Multiprogramming**: Multiprogramming is a OS technique where multiple programs are kept in main memory, and the CPU switches to another program when the current one waits for I/O, to maximize CPU utilization.

⇒ Why it's needed:
- CPU is very fast
- I/O devices are slow
- Without multiprogramming:
  - CPU stays idle during I/O
- With multiprogramming:
  - CPU always has some work.

⇒ Behind the scene:
1. Several programs are loaded into RAM
2. One program is given the CPU
3. Program requests I/O (disk, keyboard).
4. Program goes to waiting state.
5. OS switches CPU to another ready program.
6. CPU keeps working, not idle.

→ only one program executes at a time (single CPU).

* **Multitasking**: Multitasking is an OS technique where the CPU time is divided into small time slices and shared among multiple tasks, giving the user the feeling that tasks are running at the same time.

12

⇒ For clarification :

- A "program" = File on Disk
- When it starts running → It becomes a process
- multitasking switches b/w these processes.

→ So the task is a process or sub process (thread) which is running

⇒ Why It's needed :

- User wants to run many programs together.
- Example : browser + music + editor.
- OS must respond quickly to user actions.

⇒ Behind the Scene :

1. multiple tasks are in ready state
2. CPU give a time slice to task 1.
3. Time slice expires
4. OS performs context Switch.
5. CPU gives time slice to task 2.
6. This repeats very fast.

only one task runs at a time on a single CPU, but it appears simultaneous.

* Multiprocessing : multiprocessing is an os technique in which two or more cpus work simultaneously to execute multiple processes, achieving true "parallel execution".

→ Multiprocessing systems have three main advantages :

(i) increased throughput : By increasing the number of processors, we expect to get more work done in less time. but if there are n number of processes the throughput will be < n.

(ii) Economy of Scale : multiprocessors systems can cost less than equivalent multiple single processes systems, because they can share single resources like storage and power supplies.

(iii) increased reliability : If functions can be distributed properly among several processors, then the failure of one processor will not halt the system only slow it down.

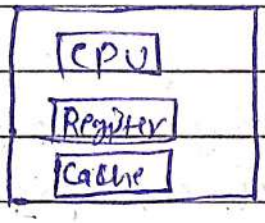
⇒ Behind the Scenes :

1. system has multiple cpu / cores
2. os divides processes among cpus
3. each cpu executes a process at the same time.
4. processes may share :
   - Memory
   - I/o devices
5. os synchronizes cpus to avoid conflicts.

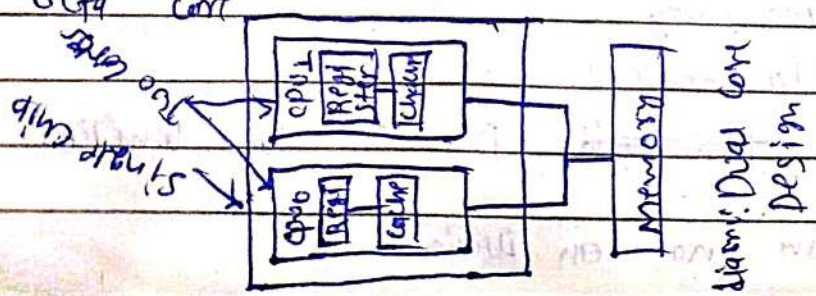
→ This is real parallelism, not an illusion.

* Core : A core is an independent processing unit inside a CPU that can execute instructions on its own.
→ a single core can execute only one instruction stream at a time.

⇒ Behind the Scene:
- one CPU chip can have multiple cores
- each core has:
  - It's own registers
  - It's own execution unit
  - It's own cache memory.
- They may share "main memory (RAM)".
- OS treats each core like a separate CPU.
- Scheduler assigns different processes / threads to different cores.



Processor	Number of cores (in a chip).
• Dual core	2
• Hexa core	6
• Octa core	8



* Types of multiprocessing: There are mainly two types:

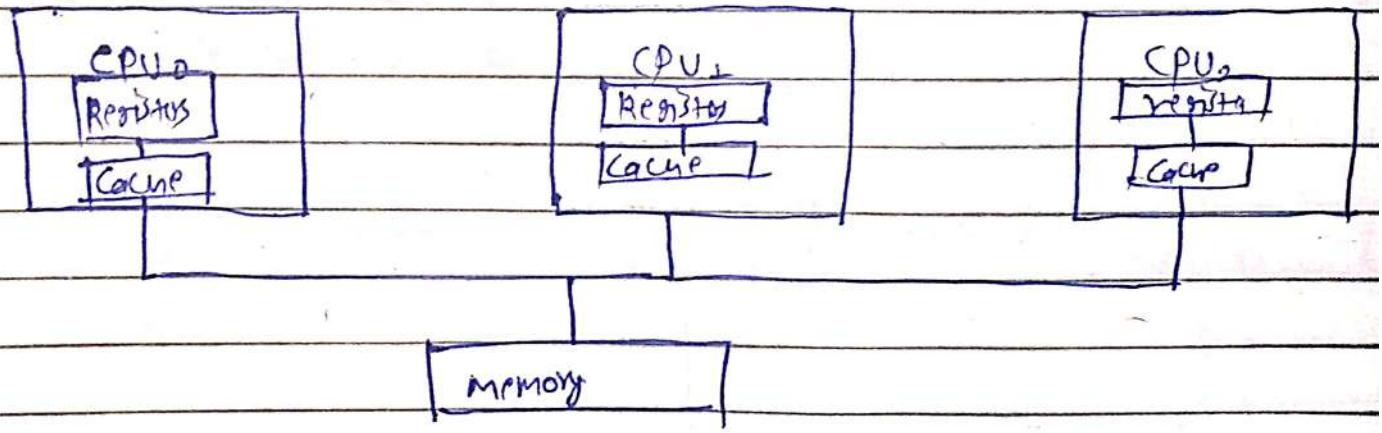
(1) Symmetric Multiprocessing (SMP): Symmetric multiprocessing is a system where all CPUs (or cores) are equal and share the same memory.

⇒ How processes are executed in SMP:

1. All CPUs run the same OS kernel.
2. processes are placed in a common ready queue.
3. The scheduler:
   - picks a ready process
   - Assign it to any free CPU.
4. Any CPU can:
   - run user processes
   - execute kernel code
5. CPU cordinate using locks and synchronization.

⇒ Why smp is efficient:

- Better Load balancing
- Higher CPU utilization
- Flexible Scheduling



⇒ Symmetric multiprocessing Architecture

Q) ~~Asy~~ Asymmetric multiprocessing : ~~Asy~~ Asymmetric multiprocessing is a system where one cpu is the "master" and the others are "slave cpu's".

⇒ How processes are executed in AMP :

1. "master cpu" runs the os.
2. master cpu :
   - schedules processes
   - Assign specific tasks to slave cpus
3. slave cpus :
   - executes only assigned tasks
   - Do not schedules processes.
4. Communication is done via message or signals.

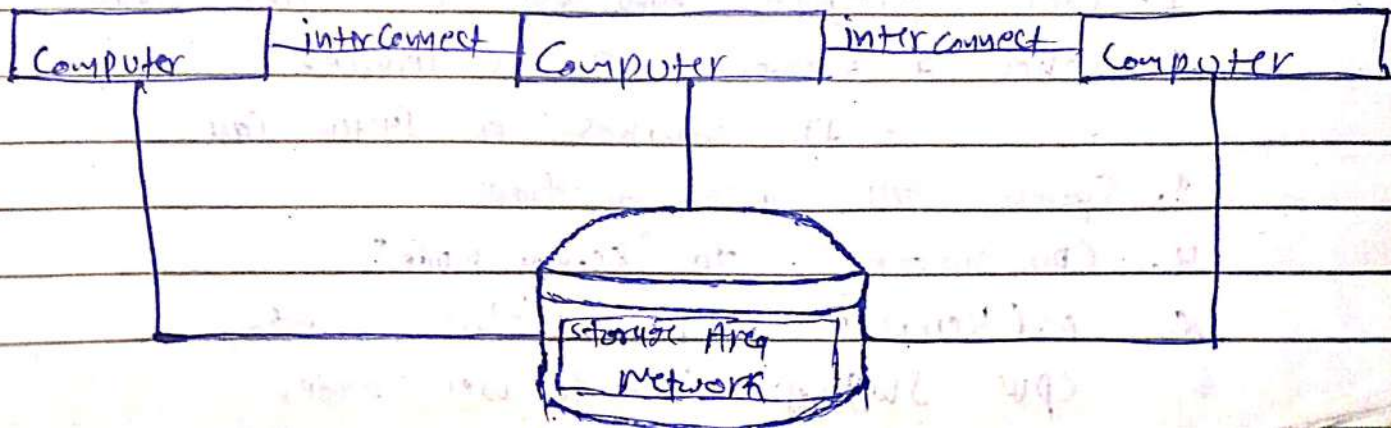
→ only master cpu controls process execution.

* Clustered System: Clustered System is a group of independent Computers (nodes) that are Connected together and works as a singe System.

⇒ Why it's needed :
- To improve availability (System keeps running if one fails).
- To improve performance
- To support Scalability
- To provide fault tolerance.

⇒ Behind the scenes (how it works):
- Each Computer (node) has :
  - its own cpu
  - its own memory
  - Its own OS
- Nodes are connected via a high-speed network.
- They may Share :
  - Storage (Common disk)
  - or communicate via message.

→ Working :
1. Tasks are distributed among nodes.
2. if one node fails :
   - Another node takes over



* Dual mode operation : Dual mode operation is mechanism in a operating system the divides Cpu execution into two modes - user mode and kernel mode - to protect the system and hardware.

→ user mode : it is the mode in which normal programs run with limited access to the system (or hardware).

→ kernel mode : It is the mode in which the operating system runs with full access to hardware and system resources.

⇒ Why it's needed :

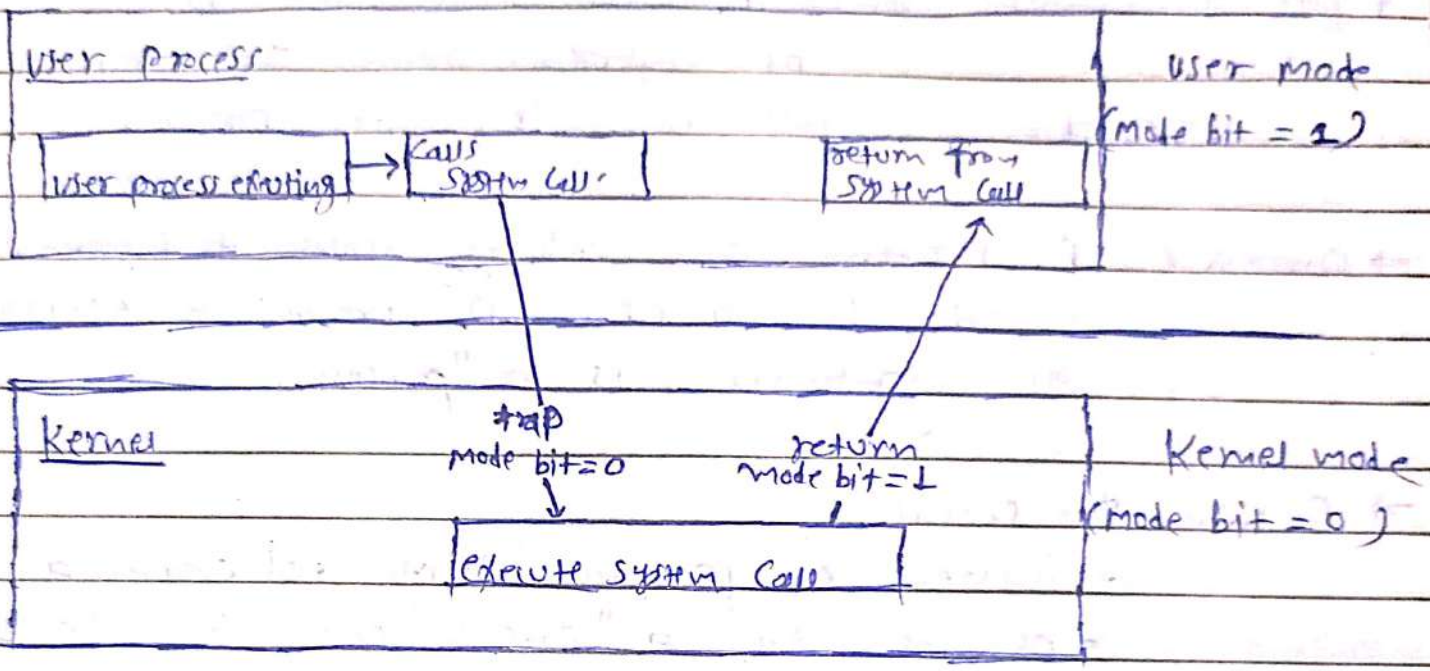
- User program should not access hardware directly.
- A mistake in user program "should not crash the whole system".
- Dual mode provides "security and stability".

⇒ Behind the scenes (how it works) :

- The cpu has a mode bit :
  - "0" → kernel mode
  - "1" → user mode

→ Flow :

1. User programs run in "user mode"
2. when a program needs os service :
   - It makes a system call.
3. System call cause a trap.
4. Cpu switches to "kernel mode"
5. OS (kernel) performs the operation.
6. Cpu switches back to user mode.


User process
User process executing
Calls System Call
return from System Call
User mode
(mode bit = 1)
Kernel
trap
mode bit = 0
return
mode bit = 1
Execute System Call
Kernel mode
(mode bit = 0)

* Process Management : Process management is the function of operating system that creates, schedules, controls and terminate processes.

⇒ Process : A program does nothing unless its instructions are executed by a CPU. A program in execution, as mentioned, is a "process".

⇒ Behind the scenes :
- When a program starts → OS creates a process.
- OS maintains a "process control block (PCB)" for each process.
- OS keeps processes in different state :
  - Ready
  - Running
  - Waiting
- Scheduler selects a process for CPU
- OS performs Context Switching when changing processes.
- When process finishes → OS terminates it and free resources.

Process management activities :
- Creating and deleting both user and system process
- Suspending and resuming processes.
- Process Synchronization
- Process Communication
- deadlock handling

* Memory Management : Memory management is the function of the operating system that controls the allocation, use and deallocation of main memory (RAM) among processes.

⇒ Why it's needed :
- RAM is limited
- Many processes need memory at the same time
- OS must :
  - Gives each process required memory.
  - prevent one process from using another's memory.
  - use memory efficiently

⇒ Behind the Scenes (how it works) :
- OS keeps track :
  - which parts of memory are free
  - which parts are occupied
- when a process starts :
  - OS allocates memory
- During execution :
  - OS protects process memory.
- when process ends :
  - OS free memory for reuse
- uses techniques like :
  - paging
  - Segmentation
  - virtual memory

* Storage management : Storage management is the function of the operating System that organizes, Stores, retrieves, and manages data on Secondary Storage devices (like HDD, SSD).

It uses :

- File Systems
- Disk Scheduling
- Space allocation methods

and all the managements are same like striyy and other Stuff:

* MASS - Storage Management : Mass storage management is a subset of storage management that specifically handles large capacity Storage devices.

* Protection and Security:

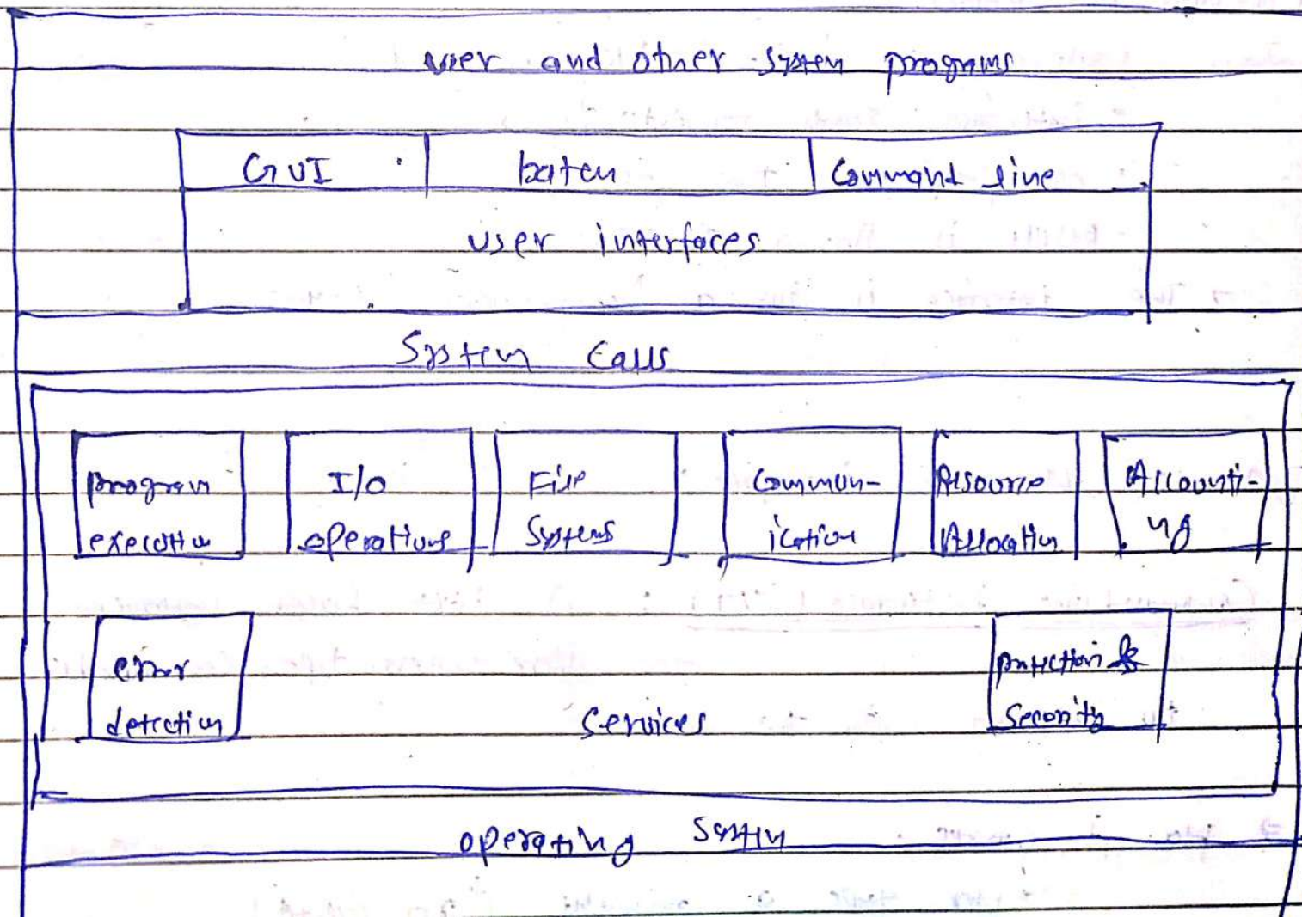
→ Protection is the mechanism that controls access to system resources

→ Security is the mechanism that defends the system against unauthorized access and attacks.

Simple line:

protection controls access, security prevents misuse.

* operating system services:
  - User Interface
  - program execution
    - Loading program in memory
    - Run it and end it properly.
  - I/O operations
    - Handles I/O ~~operation~~ devices b/w programs.
  - File System manipulation
  - process Communication
  - Error Detection
  - Resource Allocation
  - Accounting
  - protection and Security.



* User operating - System Interface : The user operating system interface is the way a user interacts with the operating system to give commands and get results.

⇒ Why it's needed :
- User cannot talk to OS directly
- interface makes OS easy to use
- Helps users :
  - Run programs
  - Manage files
  - Control the system.

⇒ Behind the scenes :
- User gives input (clicks / commands)
- interface sends requests to OS
- OS performs the operation
- Result is shown to the user

→ The interface is OVS a "communication layer.

? Types of user - OS interface :

1) Command line Interface (CLI) : A text based interface where users type commands to interact with the OS.

⇒ How it works :
- User types a command ("rm file.txt")

- Command is sent to the "os shell"
- Shell converts it into "system call".
- os executes the request.

⇒ why its used:

- Very powerful and fast
- uses less memory
- prefered by programmer and system admins.

examples: - widows Command prompt
- Linux terminal
- Powershell

(2) graphical user interface = A visual interface using windows, icons, menus and mouse.

⇒ How it works:

- user clicks or drags
- GUI sends event to os
- os performs the action
- Result is shown visually.

⇒ why it's used:

- easy to learn
- user-friendly
- no need to remember commands.

⇒ examples: - windows
- macOS
- Linux desktop environments.

* System calls: A system call is a mechanism that allows a program to request services from the operating system (kernel).

⇒ why it's needed:
- User programs run in user mode.
- They cannot access hardware directly.
- System calls provides a safe way to:
  - Access files
  - use devices
  - Create processes
  - Use memory

→ Without system calls → application cannot use OS services.

⇒ Behind the scenes:
1. Application runs in user mode
2. Application needs an OS service (e.g. read file).
3. It makes a system call which cause a trap
4. CPU switches to kernel mode
5. Kernel perform the requested operation.
6. Control returns to user mode.

Example: Copying one file's data into another file:

→
- program runs in user mode
- Calls "open()" for source file.
- Trap occurs → CPU switches to kernel mode
- Kernel checks availibility, existence, permission and return the descriptor

- Calls "open()" for destination file
  - Kernel creates / opens file and return file descripter
- Calls "read()" on source file.
  - Trap → Kernel mode
  - Kernel asks disk drive to read the data
  - Disk controller reads data and ~~return~~ sends hardware interrupt
  - Kernel places data in buffer. (in small chunks).
- Calls "write()" to destination file.
  - Trap → Kernel mode
  - Kernel sends data to disk driver
  - Disk controller writes data and sends interrupt
- "read()" and "write()" repeat until file ends.
- Calls "close()" for both files.
  - Kernel releases resources.

⇒ Types of System Calls :

(i) Process Control :
- Create processes, terminate process
- Load, execute.
- get process attributes, set process attributes
- wait event, signal event.
- Allocate and free memory

(ii) File management :
- Create file, delete file

- open, close
- Read, write, Reposition
- get file attribute, set file attribute

(iii) Device management :

- Request device, release device
- Read, write, Reposition
- get device attribute, set device attribute
- Logically attach or detach devices

iv) information maintenance :

- get time or date, set time or date
- get system data, set system data
- get process, file, or device attributes.
- set process, file, or device attributes.

v) Communication :

- Create, delete Communication Connection
- Send receive messages
- transfer Status information
- attach and detach remote devices.

vi) Protection :

- get file Permissions
- set file permissions

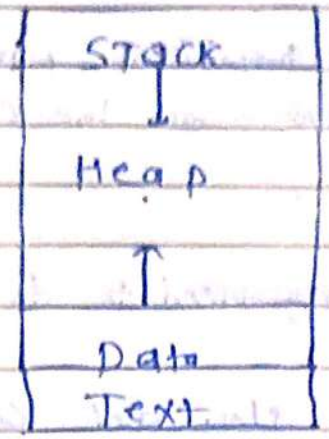
Chapter - 3

* Process: "A process is a program that is currently running in the computer." or
"A process is an active instance of a program" or
"A process is an program in execution."

⇒ Why it's needed: • A program on disk is inactive.
• To actually do work:
• It must be loaded into memory
• It must use CPU
• OS needs a way to:
• Run many programs together
• Manage CPU and memory properly.
→ The "Way" is a process.

⇒ Behind the scene:
• program is stored on disk.
• when you run it:
• OS loads it into RAM
• OS Assigns:
• CPU time
• Memory
• Resources
• OS Creates a process control block (PCB) for it.
• Now the program becomes a process.
• OS switches CPU between processes using context switching.

→ process in memory :



(i) Text (Code Segment) :
- Stores the actual program instructions.
- Compiled machine code
- Read only.
- Same code can be shared by multiple processes.

Example: instructions of main, loops, functions.

(ii) Data Segment :
- Stores global and static variables
- Exists for the entire lifetime of process

(iii) Heap :
- Stores dynamically allocated memory.
- Used when program requests memory at run time.
- grows upward.
- memory must be manually freed.

(iv) Stack : - Stores :
- function calls
- Local variables
- return addresses

- grows downward and automatically managed

42 Important: OS usually schedules threads, not whole processes.

---

* Thread: A thread is a smallest unit of execution inside a process. or
"A Thread is a lightweight part of a process that actually runs on the CPU.

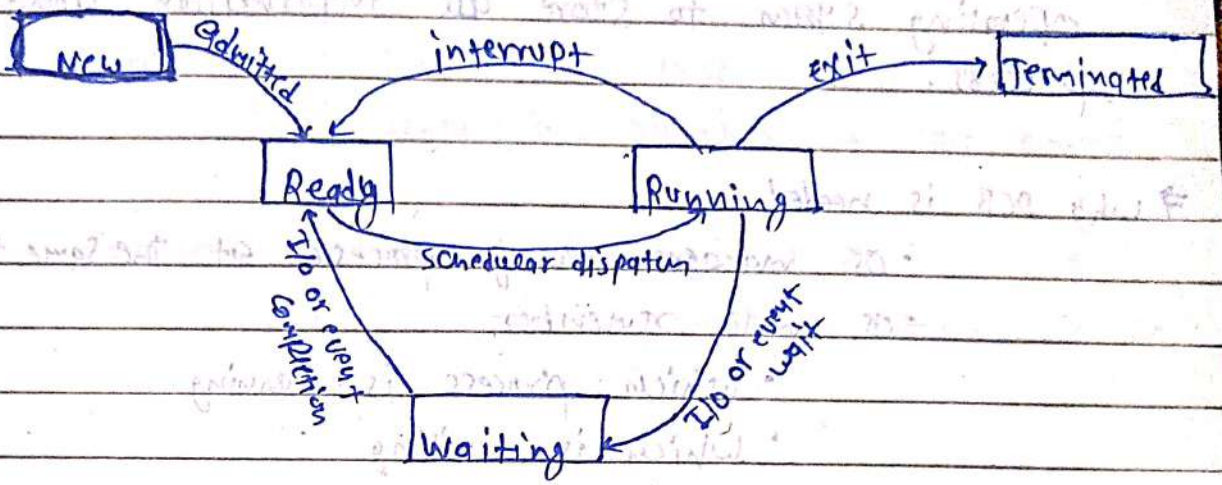
⇒ Why Threads are needed:
- A single process may need to do many thing at the same time.
- Creating processes is slow and costly
- Threads:
  - Are faster to create
  - Use less memory
  - improve performance.

⇒ Behind the scene:
→ Inside one process:
- All threads share:
  - Code (text)
  - Data
  - Heap
  - open files
- Each thread has its own:
  - Stack
  - Register
  - program Counter

process → (first Thread is known as "main" Thread)
- Thread 1 → own stack
- Thread 2 → own stack
- Thread 3 → own stack

— shared code, data, heap.

* Process States:



(i) ~~the~~ New:

- process is being created
- PCB is created and memory is allocated

(ii) Ready:

- process is in "main memory".
- Ready to run waiting for "cpu time".

(iii) Running:

- process in currently executing on cpu

(iv) Waiting:

- process is waiting for an event.
- example:
  - I/O completion
  - ~~the~~ user input

(v) Terminated:

- process has finished execution
- Resources are ~~re~~ released

44

* Process Control Block : A process Control block (PCB) is a data structure used by the operating system to store all information about a process.

⇒ Why PCB is needed :

- OS manages many processes at the same time
- OS must remember
  - which process is running
  - which is waiting
  - what resources a process is using.
- PCB helps os track and control each process.

PCB Diagram :

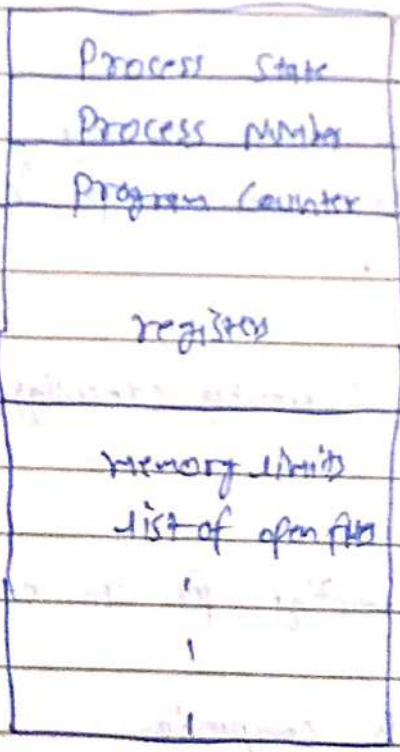
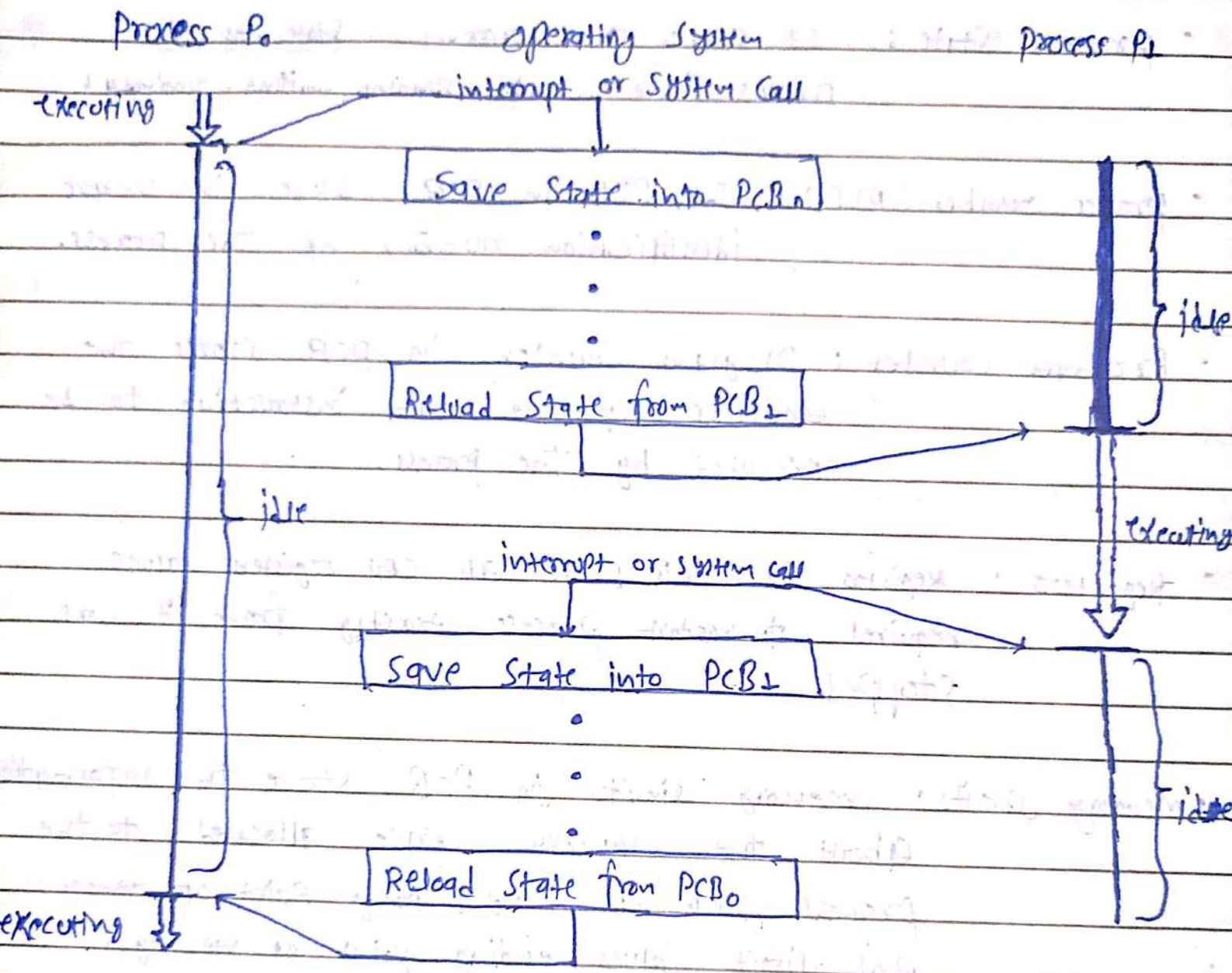


figure: Process Control blocks.

- Process State : It stores the current state of the process (new, ready, running, waiting, terminated).
- process number (PID) : The PID in PCB stores the unique identification number of the process.
- Program Counter : program Counter in PCB stores the address of the next instruction to be executed by the process.
- Registers : Registers in PCB stores all cpu register values required to restart process exactly from it was stopped.
- Memory limits : memory limits in PCB store the information about the memory space allocated to the process, such as base (starting point of memory) and limit values (ending point of memory).
- List of open files : The list of open files in PCB stores information about all files currently opened by the process.


Process P_0
Operating System
Process P_1
executing
interrupt or System Call
Save State into PCB_0
idle
Reload State from PCB_1
idle
executing
interrupt or system call
Save State into PCB_1
idle
Reload State from PCB_0
executing

*Scheduling queues: Scheduling queues are lists of processes waiting for different resources or execution stages in the operating system.

⇒ Types of Scheduling queues:

(i) Job queue: Stores all processes in the system, including those waiting to enter main memory.

(ii) Ready queue: Stores processes that are in main memory, and ready to execute, waiting for CPU.

(iii) Waiting queue: Stores processes waiting for I/O operations or events.

(iv) Device queue: Store processes waiting for a specific device (printer, disk, etc).

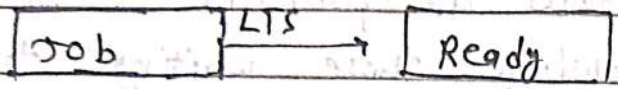
*Schedulers: Schedulers are OS components that decides which process should be executed and when.

⇒ Why Schedulers are needed:

- Many processes exist at the same time.
- CPU and memory are limited
- Schedulers help:
  - Select processes
  - Control process flow
  - improve system performance.

⇒ Types of Schedulers:

(i) Long-term Scheduler (Job Scheduler): This Scheduler selects process from job queue (secondary memory) to ready queue (primary memory):



→ The long term scheduler Controls the "degree of multiprogramming". Meaning it manages the number of process can present in the ready queue to be executed.

- Works:
  - Less frequently
  - slower.

(ii) Short-term Schedulers (CPU Scheduler): The short term Scheduler selects process from ready queue and assign CPU to executed.

- purpose:
  - Decides which process runs next
  - improves CPU utilization

- works:
  - very frequently
  - very fast

(iii) Medium term Schedulers: Some operating system such as time sharing systems, may introduce an additional, intermediate level of Scheduling.

- It removes processes from memory temporarily and brings them back later.

→ The process of removing processes from memory temporary and bringing the back later is known as "swapping".

⇒ When it works:

- Too many processes in RAM
- OS moves some unnecessary process / processes into disk

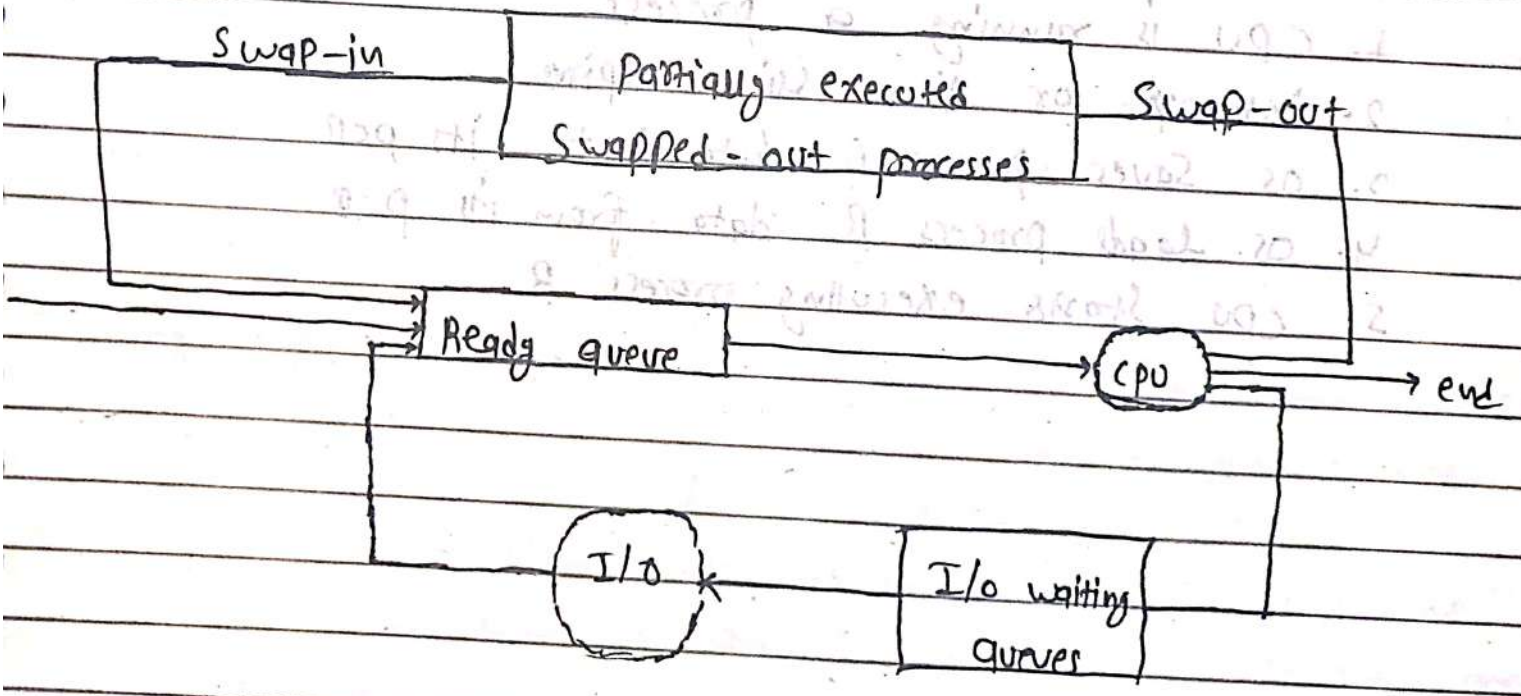


fig: Addition of medium-term scheduler to the queueing diagram

So

* Context Switch : Context Switching is the process of saving the state of the current process and loading the state of another process so the CPU can switch execution.

⇒ Why it's needed :
- CPU runs only one process at a time (per core).
- Many processes exists simultaneously.
- OS must switch CPU b/w processes.

→ The "context" means the current ~~information~~ execution information

⇒ Behind the scene (how it actually happens) :
1. CPU is running a process
2. interrupt or time slice expires
3. OS saves process A data into its PCB
4. OS loads process B data from its PCB
5. CPU starts executing process B

* Types of processes (based on Resource usage):

i) cpu bound process: A cpu bound process is a process that spends most of its time performing calculations using the cpu. (high "cpu burst").

→ Characteristics:

- Requires heavy computation
- Uses cpu more than I/o
- Runs continuously with fewer waiting periods.

ii) I/o bound process: An I/o bound process is a process that spends most of its time performing input/output operations.

→ Characteristics:

- frequently waits for devices
- uses cpu for short time
- More waiting state.

⇒ Why we need good "process mix": ~~we always~~ LTS should always have a good process mix cuz if all processes are I/o bound, the ready queue will always be empty and the STS will have a little to do. if all processes are cpu bound, the I/o waiting queue will be almost empty, devices will go unused, and again the The System will be unbalanced. So the System with the best performance will thus have a good mix/combination of cpu bound and I/o bound processes.

* Child process: A child process is a new process created by another process, called the parent process.

⇒ When a child processes are created:
- When a program starts another program
- Example:
  - You open browser
  - Browser opens another tab
  - ~~the~~ new tab may run as child process.

Or - Task Division
- Break big task into smaller tasks

- ~~Para~~ parallel execution:
  - parent and child can run together

⇒ Behind the Scene: (how child process is created):
→ usually created using:
- fork()

Flow:
1. parent process calls fork()
2. OS creates new PCB for child containing parent ID (PID)
3. OS allocates new memory space
4. Child process starts execution
5. parent and child run independently.

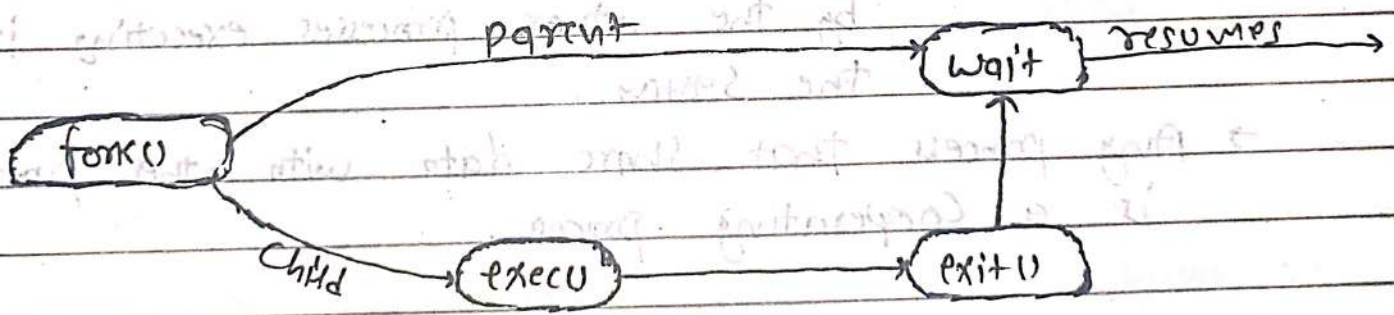
→ Relationship b/w parent and child:
- Child get copy of:
  - code
  - data
  - resources
- Parent may wait for child to finish

⇒ Zombie process: A zombie process is a process that has finished execution, but it's PCB (process entry) still exists in the system.

→ why zombie process occurs:
- when a child process finishes.
- But the parent process "has not read its exit status" or "parent got deleted".

⇒ Zombie process:
- Does not use cpu
- Does not run
- only keeps process entry.

To prevent this parent should take child's exit status or parent should get deleted after all childs are finished or terminated.



* Interprocess Communication : interprocess communication (IPC) is the mechanism that allows processes to exchange data and communicate with each other.

The process executing concurrently in operating system may be either "independent processes" or "Cooperating processes"

(i) independent process : A process is independent if it cannot affect or be affected by the other processes executing in the system.
→ Any process that does not share data with any other process is independent.

(ii) Cooperating process : A process ~~that does not~~ is Cooperating if it can affect or be affected by the other processes executing in the system.
→ Any process that share data with other processes is a Cooperating process.

⇒ The interprocess communication mostly happens b/w "Cooperating process".

There are two fundamental models of interprocess Communication :
(i) Shared memory
(ii) message passing.

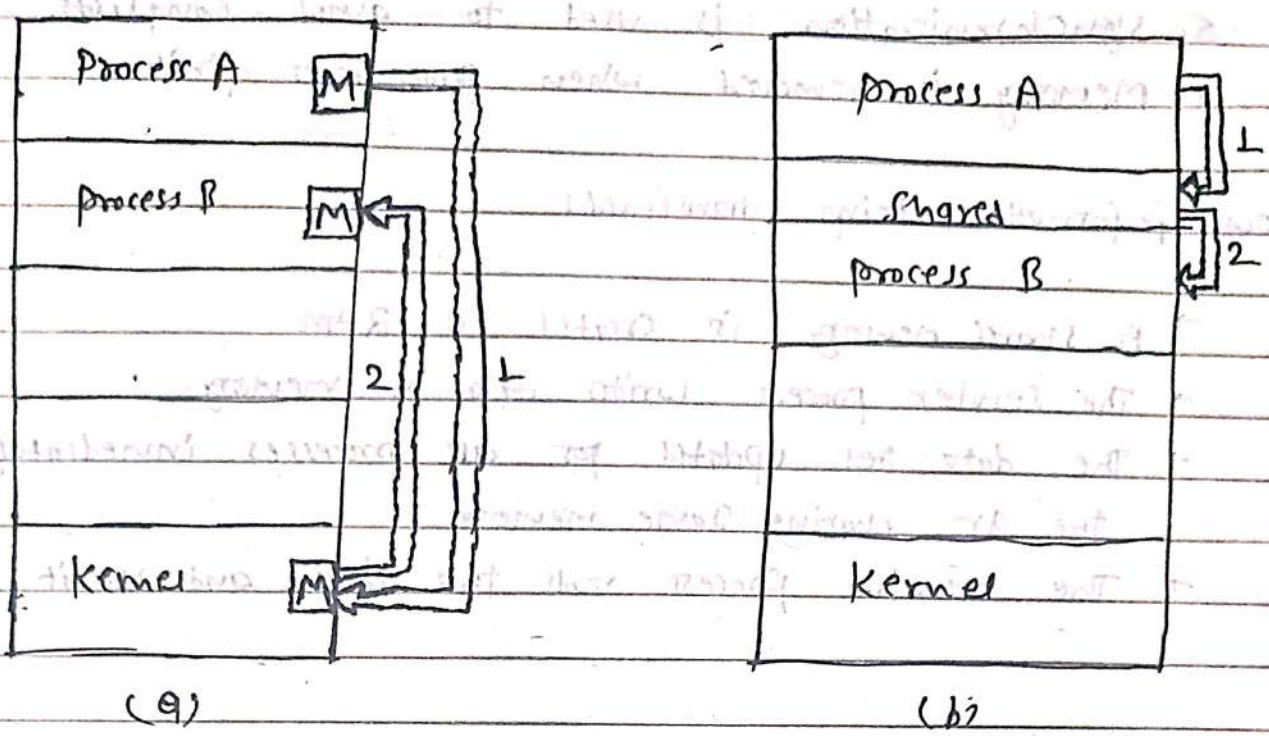


fig: Communication models : (a) message passing : (b) shared memory.

* Shared Memory : Shared memory is an IPC method where multiple processes communicate by accessing same memory region.

→ where it is placed :
- Created by OS in main memory (RAM)
- It is a common memory area, not private to any process.

⇒ How it works :
1. A process request shared memory
2. OS creates shared memory block in RAM
3. Other processes attach to that memory.
4. processes read and write data directly.

5. Synchronization is used to avoid conflicts.
6. Memory is removed when processes finish.

⇒ How information being transfferd:

→ A shared memory is created in RAM
→ The sender process writes data in memory.
→ The data get updated for all processes immediately cuz the are sharing same memory.
→ The receive process reads the data and use it.

* Message Passing: Message passing is an IPC method where processes communicate by sending and receiving message through the operating system.

⇒ Why message passing is needed:
- processes may have separate memory.
- processes can be from distributed computers.

⇒ Where message passing happens:
- messages are handled by the operating system kernel.
- OS manages message transfer b/w processes.

⇒ How message passing works:
1. Sender creates a message
→ message usually contains:
- Sender PID
- Receiver PID
- Data.

2. Sender sends message using system call:
   - send()
   → OS stores message temporarily.

3. Receiver request message:
   - receive()
   → OS delivers message to receiver

4. Communication Complete, Data safely transffered.

⇒ Blocking and non-blocking Message Passing:

- Blocking message passing means the sender or receiver waits until the message operation is completed.

  → How it works:
  - Blocking send:
    - Sender sends message
    - Send waits until receiver receives message
  - Blocking receive:
    - Receiver waits until message arrives.

- Non-blocking message passing means the sender or receiver continues execution without waiting.

  → How it works:
  - Non blocking send:
    - Sender sends message
    - Sender continues execution immediately
  - Non-blocking receive
    - Receiver checks message
    - if msg is not available
      → Continues execution.

58

* buffering : Buffering is the temporary storage of data/ message in memory during communication b/w processes.

⇒ why buffering is needed:
- Sender and receiver may work at different speed.
- buffer stores message until receive is ready.
- prevents data loss.
- improves communication efficiently.

⇒ where buffer is placed:
- placed in RAM manged by os.

⇒ Types of buffering:
(i) Zero Capacity buffer (no buffer) **
- buffer size = 0
- Sender must wait until receiver is ready

(ii) Bounded Capacity Buffer:
- Buffer has limited size
- Sender can send message until buffer is full.

(iii) Unbounded capacity buffer:
- buffer has unlimited size
- Sender never waits.
- message stored until receiver reads them.

Chapter - 4 : CPU Scheduling.

* CPU Scheduling : CPU Scheduling is the process of deciding which process gets the CPU and for how long.

⇒ Why CPU Scheduling is needed :
- Many processes are ready to run.
- CPU can execute only one process per core.
- OS must :
  - Share CPU properly
  - improve system performance
  - Reduce waiting time

Without CPU Scheduling → Chaos Starvation

⇒ Behind the Scenes (how it actually works) :
1. process enters ready queue
2. Scheduler selects one process
3. Dispatcher gives CPU to selected process
4. Process runs
5. CPU is taken back (interrupt / time slice).
6. Scheduler selects next process.

⇒ Key goals of CPU Scheduling :
- Maximize CPU Utilization
- Maximize throughput
- ~~Maximize~~ Minimize waiting time
- Minimize turnaround time
- Minimize response time
- fairness.

⇒ when cpu scheduling happens :

→ CPU scheduling occurs when a process :

Non-preemptive:
- Moves from running → waiting
- Moves from running → Ready
- Moves from waiting → Ready
- Terminates.

Preemptive: Moves from running → Ready; Moves from waiting → Ready

---

* Scheduling Criteria : Scheduling criteria are the standards used to evaluate the performance of CPU scheduling algorithms.

~~Mark~~ main scheduling criteria :

(1) CPU Utilization :

- percentage of time CPU is busy
- Higher CPU utilization = better performance

Goal : maximize.

(2) Throughput :

- Number of processes completed per unit time.

Goal : Maximize

(3) ~~Turn~~ Turnaround time :

- total time taken by a process from Arrival to Completion.
- Goal : Minimize

(4) Waiting time :

- total time a process spends waiting in Ready queue
- Goal : minimize

(5) Response time :

- Time from process arrives to first CPU response.
- Goal : minimize

⇒ Why these criteria are important :

- they help compare algorithms
- Decide which algorithm is better for :
  - Batch systems
  - Interactive systems
  - Real-time systems.

* CPU scheduling Types :

(1) Non-preemptive scheduling : in non-preemptive scheduling, Once a process gets the CPU, It keeps the CPU until it finishes or goes into waiting state.

⇒ How it works :

- process starts executing
- CPU is not taken away
- Switch happens only when :
  - process completes or
  - process request I/O

Examples :

- FCFS (first come first serve).
- Non-preemptive SJF

: Advantages:
- Simple to implement
- No frequent context switching

: Disadvantages:
- Poor Response time
- Long process can block others.

(2) Preemptive Scheduling: in preemptive scheduling, the os can take CPU away from a running process.

⇒ How it works:
- process is running
- CPU may be taken away if:
  - Time slice expired
  - Higher priority process Arrives.
- Context switch occurs

⇒ Examples:
- Round Robin
- Preemptive SJF (Sorted Remaining time first).
- priority scheduling (preemptive)

⇒ Advantages
- Better response time
- good for interactive systems

⇒ dis advantages
- More Context switching overhead
- More complex.

Scheduling Algorithms

(1) First Come first serve (FCFS): FCFS is a non-preemptive CPU scheduling Algorithm in which the process that arrives first gets the CPU first.

⇒ How it works:
- Processes are placed in Ready queue
- CPU is allocated in Arrival order
- Once a process gets CPU, it runs till completion or waits for I/O

- Advantages:
  - Very simple
  - Easy to implement
  - No starvation
- Disadvantages:
  - High waiting time
  - Convoy effect (one long process makes others wait)
  - ~~Poor~~ Poor response time.

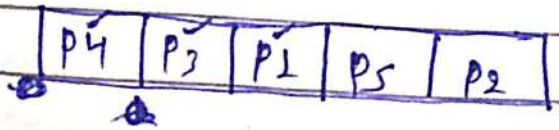
⚠ It uses simple FIFO queue, no priority, no time slicing.

Example:

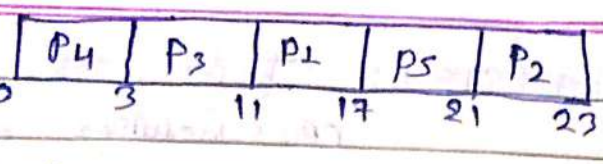
process	Arrival - time	CPU Burst time
P_1	2	6
P_2	5	2
P_3	1	8
P_4	0	3
P_5	4	4

Solution:

Ready queue:



Gantt chart :



→ Waiting time : $P_1 = 11 - 2 = 9$

(Starting time - Arrival/last Completion time) $P_2 = 21 - 5 = 16$

$P_3 = 3 - 1 = 2$

$P_4 = 0 - 0 = 0$

$P_5 = 17 - 4 = 3$

Avg. waiting time = $\frac{9+16+2+0+3}{5} = \frac{30}{5} = 6$ millisec.

→ Turnaround time : $P_1 = 17 - 2 = 15$

~~(Priority time~~ $P_2 = 23 - 5 = 18$

(Completion time - Arrival time) $P_3 = 11 - 1 = 10$

$P_4 = 3 - 0 = 3$

or (waiting time + Burst time) $P_5 = 21 - 4 = 17$

Avg. Turnaround time = $\frac{15+18+10+3+17}{5} = \frac{63}{5} = 12.6$

⇒ If the ready queue is empty at any moment CPU will be ideal until a process comes in Ready queue

Throughput = $\frac{5}{23}$ = 0.21 process / millisec.

(2) Shortest Job First (SJF): Shortest Job first is a CPU scheduling Algorithm that selects the process with the smallest CPU burst time first which has arrived

in one line:

"SJF schedules the process with the shortest time first"

⇒ Type:

- Can be:
  - Non-preemptive SJF
  - Preemptive SJF (Shortest remaining time first = SRTF).

Non ~~[illegible]~~ preemptive SJF:

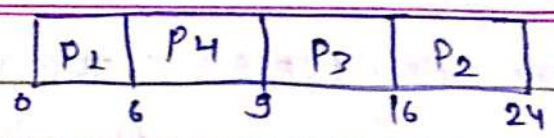
- Advantage:
  - Gives minimum avg waiting time (mathematically proven)
- Disadvantage
  - May cause Starvation
  - How to predict burst time.

Example: SJF (non-preemptive):

processes	Arrival time	Burst time
P_1	0	6
P_2	1	8
P_3	2	7
P_4	3	3

Ready queue → | P_1 | P_2 | P_3 | P_4 |

Gantt chart :



Waiting time : $P_1 = 0 - 0 = 0$

$P_2 = 16 - 1 = 15$

$P_3 = 9 - 2 = 7$

$P_4 = 6 - 3 = 3$

$$\text{Avg WT} = \frac{0+15+7+3}{4} = \frac{25}{4} = 6.25$$

TAT : $P_1 = 6 - 0 = 6$

$P_2 = 24 - 1 = 23$

$P_3 = 16 - 2 = 14$

$P_4 = 9 - 3 = 4$

$$\text{Avg TAT} = \frac{6+23+14+4}{4} = 11.75$$

$$\text{Throughput} : \frac{5}{24} = 0.20 \text{ process/millisec}$$

* Shortest Remaining time First (SRTF) : This is also known as preemptive SJF where the process with the Smallest Remaining CPU burst time is selected for execution from the Ready queue.

⇒ How it works:

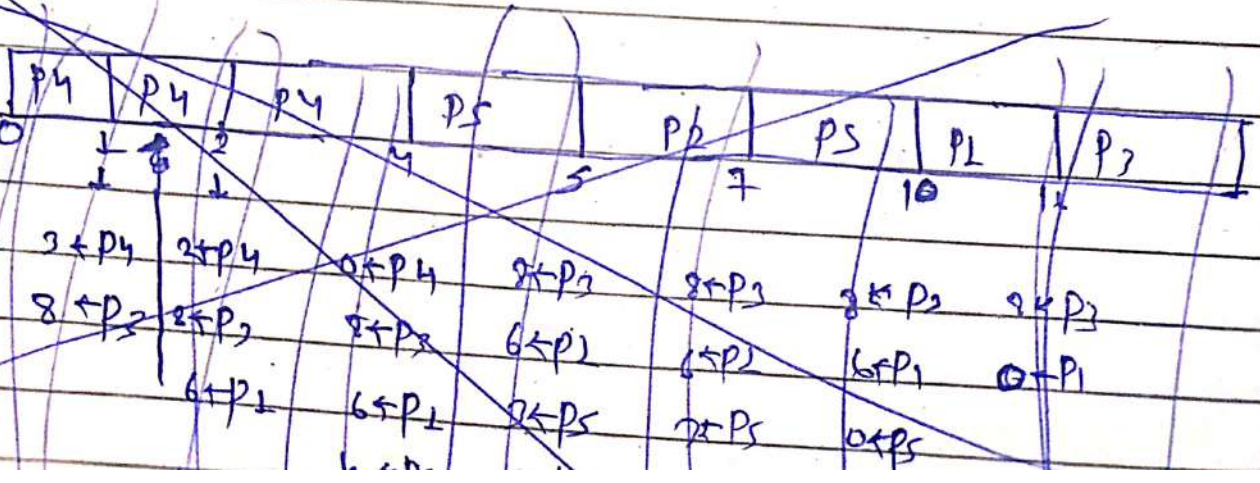
1. At any time, check all ready processes.
2. Select the one with smallest Remaining time
3. if a new process arrives with smaller Remaining time :
   - Stop current process
   - Perform context switch
   - Run new shorter process.

~~Preemption~~

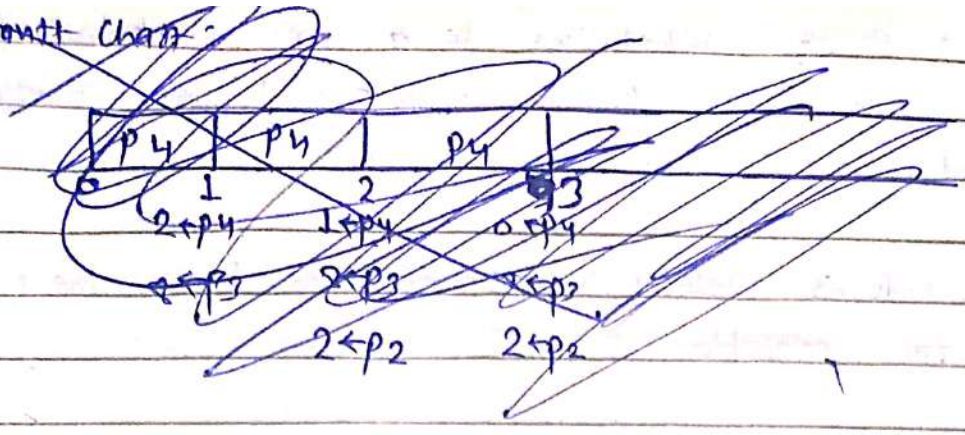
Example :

process	Arrival time	Burst time
P1	2	6
P2	5	2
P3	1	8
P4	0	3
P5	4	4

Solution : Gantt chart :



Gantt Chart:



Gantt Chart :

P4	P4	P4	P1	P5	P2	P5	P1	P3
0	1	2	3	4	5	7	10	15

P4→3 P4→2 P4→1 P4→0 P3→8 P3→8 P3→8 P3→8 P3→8 P3→
P3→8 P3→8 P3→8 P1→5 P1→5 P1→5 P1→5 P1→0
P1→6 P1→6 P5→4 P5→3 P5→3 P5→0
P2→2 P2→0

Waiting time = P1 = (3-2) + (10-4) = 7
P2 = 5-5 = 0
P3 = 15-1 = 14
P4 = 0-0 = 0
P5 = (4-4) + (7-5) = 2

→ 23/5 = 4.6

TAT = (waiting time + Burst time)

P1 = 7 + 6 = 13
P2 = 0 + 2 = 2
P3 = 14 + 8 = 22
P4 = 0 + 3 = 3
P5 = 2 + 4 = 6

→ 46/5 = 9.2

Throughput = 5/27 = 0.21 process/millisec

* Priority Scheduling : priority Scheduling is a cpu scheduling where the process with the highest priority executed first.

From line :

"priority scheduling selects the process with the highest priority for execution".

⇒ How priority is Assigned :

- Each process is assigned a priority number
- Can be :
  - Static (fixed)
  - Dynamic (Changes during execution)

⚠ Important :

- Sometimes smaller Number = higher priority
- Sometime larger Number = higher priority

Types of priority Scheduling :

→ Non-Preemptive priority :

: once process starts → runs until completion

→ Preemptive priority :

- If higher-priority process arrives, current process is preempted.

Advantages

- Important tasks executed first
- Good for real-time system

- Disadvantages
- Starvation (low priority processes may never execute)

⇒ Solution to starvation :

→ Aging :

- Gradually increase priority of waiting processes
- prevents starvation.

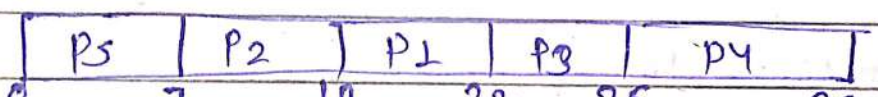
Example :

process	Arrival time	Burst time	priority
P1	6	12	3
P2	4	3	1
P3	8	4	3
P4	2	3	4
P5	0	7	2

→ Smaller number implies higher priority

Solution : (preemptive) :

Grantt :



WT : P1 = 10 - 6 = 4
P2 = 7 - 4 = 3
P3 = 22 - 8 = 14
P4 = 26 - 2 = 24
P5 = 0 - 0 = 0

45/5 = 9

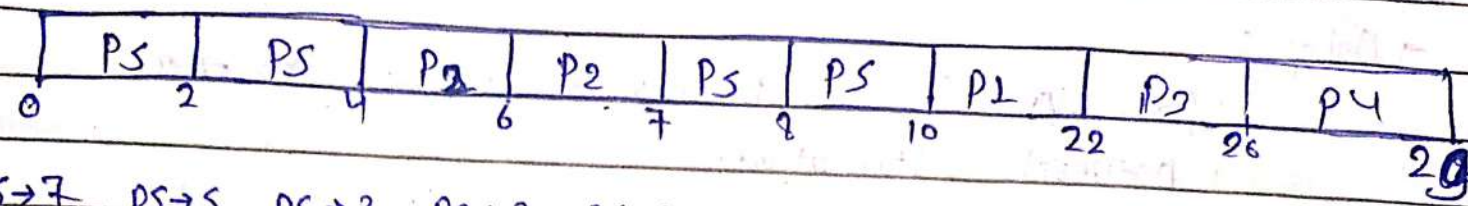
AT : (WT + BT)

P1 = 4 + 12 = 16
P2 = 3 + 3 = 6
P3 = 14 + 4 = 18
P4 = 24 + 3 = 27
P5 = 0 + 7 = 7

74/5 = 14.8

Throughput = $\frac{5}{29}$ = 0.17 P/ms

Preemptive :



P5→7 P5→5 P5→3 P5→1 P5→3 P5→2 P5→0 P4→3 P4→3
P4→3 P4→3 P4→3 P4→3 P4→3 P4→3 P1→0 P3→0
P2→3 P2→2 P2→0 P1→12 P1→12 P3→4
P1→12 P1→12 P3→4 P3→4

WT : P1 = 10 − 6 = 4
P2 = 4 − 4 = 0
P3 = 22 − 8 = 14
P4 = 26 − 2 = 24
P5 = (0 − 0) + (7 − 4) = 3
→ 45/5 = 9

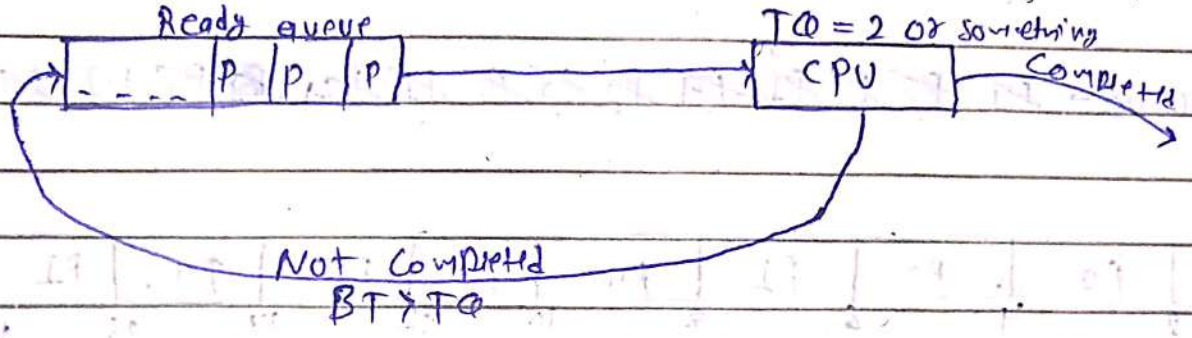
TAT = P1 = 4 + 12 = 16
P2 = 0 + 3 = 3
P3 = 14 + 4 = 18
P4 = 24 + 3 = 27
P5 = 3 + 7 = 10
→ 74/5 = 14.8

Throughput : $\frac{5}{29}$ = 0.17 P/ms.

* Round Robin (RR): Round Robin is a preemptive CPU Scheduling Algorithm where each process gets CPU for a fixed time called time Quantum (TQ).

In simple words:

"if a process does not finish within time quantum → It is preempted and moved to the back of the ready queue."



⇒ Why Round Robin is used:

- Designed for time-sharing Systems
- gives fair CPU share to all processes
- Improves Response time.

⇒ How it works:

1. processes are placed in ready queue (FIFO)
2. First process gets CPU for time quantum.
3. If finished → removed
4. If not finished → Remaining time updated
5. process goes to end of queue.
6. Repeat.

* important Concept: Time Quantum:

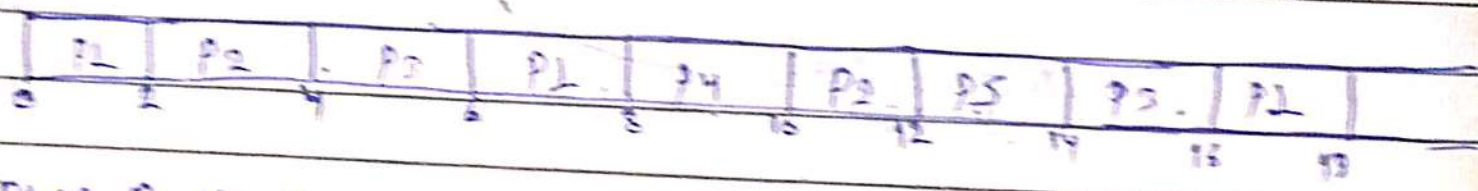
- small TQ : More Context Switching (overhead)
- Large TQ : Becomes like FCFS

so TQ size is very important

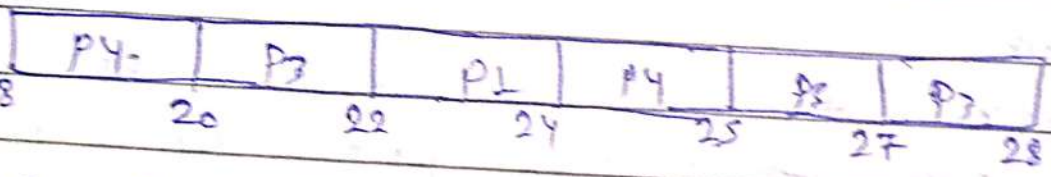
Example:

Process	Arrival time	Burst time
P1	0	8
P2	1	4
P3	2	9
P4	3	5
P5	6	2

TQ = 2



P1→8 P2→4 P3→2 P4→2 P1→2 P2→2 P5→0 P3→2 P4→3
P1→0 P2→0 P3→0 P1→0 P4→0 P2→0 P5→0 P3→0 P1→0



P3→0 P3→0 P3→0 P3→0 P3→0 P1→0 P3→0
P1→0 P1→0 P1→0 P1→0 P4→0
P4→0 P4→0 P4→0 P4→0

	CT	TAT (CT - AT)	WT (TAT - BT)
P1 = 24	24	24	16
P2 = 12	11	11	7
P3 = 28	28	26	17
P4 = 25	22	22	17
P5 = 14	8	8	6

→ 91/5 = 18.2

→ 67/5 = 12.6